

从零训一个 0.6B C++ Code LLM · 阶段一

预训练到能写基本 C++

手写 Transformer + 从头预训练 · The Stack v2 C++ · 数据/tokenizer/模型/训练机制全链路 + CE/困惑度第一性推导

本篇脉络（阶段 A：预训练 → 会写 C++）

- 0. 目标与全链路:预训练 → SFT → RL,本篇只做第一步 A
- 1. 为什么是"语文写作"到"做题"的范式:next-token 的本质
- 2. 数据:The Stack v2 C++ 清洗（去重/质量/许可/PII）
- 3. Tokenizer:BPE + C++ 关键词保护 token
- 4. 从 ID 到 embedding:one-hot、查表、端到端学表示
- 5. 模型:0.6B decoder（RoPE/SwiGLU/RMSNorm/32层深）
- 6. 训练机制:CE loss、BP 更新 embedding、梯度累积
- 7. Smoke test 的第一性推导:为什么初始 $\text{loss} = \log|\text{vocab}|$, $\text{ppl} = |\text{vocab}|$
- 8. 困惑度的直觉:"平均在几个词里选"
- 9. 显存 back-and-forth:激活、grad checkpointing、OOM 的取舍
- 10. 速度 back-and-forth:GPU 利用率、异步预取、真瓶颈定位
- 11. 结果:loss 曲线的陡降与平台期

0. 目标与全链路

目标:从零(随机初始化)训一个 **0.6B 的 decoder-only Transformer**,让它**会写 C++**——不是调用现成模型,而是数据清洗、tokenizer、模型、训练循环全部自己搭,把"先贤训 code LLM"这条路完整走一遍。

完整链路三步,本篇只做第一步:

A. 预训练（本篇）:海量 C++ 代码上做 next-token → 学"会写 C++"。

B. SFT:题解数据微调 → 对齐到"看题写解法"。

C. RL(RLVR):测试通过与否当 reward → 逼"真能 AC"。

模型规模按算力选 0.6B(A100 单卡可训);数据用 The Stack v2 的 C++ 子集清洗后约 14B token,正好落在 0.6B 的 Chinchilla 最优附近。

1. 从"语文写作"到"做题":next-token 的本质

预训练的核心是 **next-token prediction**:给前面的 token,预测下一个。它的本质是"语言建模"——学"什么样的 token 序列是自然、符合语料统计的"。

关键区分:next-token 学的是"写得像"(语言统计),不是"算得对"(逻辑正确)。`"1+1=3"`作为字符串语法通顺,next-token 不觉得有问题。所以**预训练能学会"写出合法、像样的 C++"**(语法、结构、常见模式),但不保证**"解决问题"**——那要靠后续 SFT/RL。本篇止于"会写 C++"。

对代码而言,next-token 要学的是 token 之间的**结构依赖**:括号配对、变量定义-使用、作用域、控制流。这些是"长距离精确依赖",正好是 attention 擅长的——这也是 Transformer 特别适合代码的原因。

2. 数据:The Stack v2 C++ 清洗

用 `the-stack-dedup` 的 C++ 子集(官方已做近似去重,省掉最重的 MinHash 步)。流式下载 + 轻量质量过滤,直接写 jsonl 分片。

清洗规则(用数据集自带的质量字段,比启发式准)

- 文件大小:200 字符 $< n < 1\text{M}$ (太小没内容,太大超 context)。
- 超长行: `max_line_length > 1000` 丢(压缩/minified 代码或数据 blob)。
- 平均行长: `avg_line_length > 200` 丢。
- 字母数字比例: `alphanum_fraction < 0.25` 丢(base64/二进制)。
- 许可证:只留宽松许可(MIT/Apache/BSD...)。
- 自动生成:头部含 "do not edit"/"autogenerated" 丢。

留存率 88%:因为 the-stack 本身预清洗过,脏数据少;+ 官方 dedup 版已去重。最终 5.6M 文件 → 清洗后遍历全量(6.4M 原始文件,100% 覆盖)。

Tokenize 成 token 分片(.bin)

每个 C++ 文件 tokenize 成 ID 序列,文件间加 `<eos>` 分隔,拼成一条长 token 流,按 2 亿 token 切成 `.bin` 分片(uint16,vocab<65536 够)。训练时 mmap 读,零拷贝。

工程细节:单条 `tok.encode()` 只用 1 核(250 docs/s);换 `tok.encode_batch()` 多线程吃满 32 核 → 1000+ docs/s,6 小时降到 1.5 小时。这是 Amdahl 定律的活例子——并行只加速 encode 那步,读文件/json/写盘仍单线程,所以是 4-5 倍不是 32 倍。

最终:5.6M docs → 13.99B token,70 个分片(~28GB)。

3. Tokenizer:BPE + C++ 关键词保护

底层和 NLP 一样:byte-level BPE(子词切分,标识符/字面量无 OOV)。但代码有个特点——**关键词和符号应该整体成 token,不该被 BPE 切怪**(如 `int` 不该切成 `in|t`)。

方案(混合):训 BPE 词表 + 把 C++ 关键词(`int/for/class/...`)和多字符符号(`::/->/</...`)作为**保护 token** 强制加入,禁止被拆。

- 无限的东西(标识符、字面量)→ BPE 子词覆盖(不 OOV)。
- 有限固定的东西(关键词、符号)→ 整体 token(干净)。

这解决了"纯 lexer 词表爆炸"和"纯 BPE 切怪关键词"两个极端。

训练量的坑:BPE 不需要全量 14B——最初喂太多跑了 139 分钟没完。参考 StarCoder2 的做法(在 The Stack "小子集"上训),**采样 5 万文件(几百 M token)40 秒训完**,单语言 + vocab 32K 足够。最终 vocab = 32006(32000 BPE + 特殊/保护 token)。

4. 从 ID 到 embedding:one-hot、查表、端到端学

tokenize 产出的是 **token ID 序列**(每个 token 在词表里的编号,0-32005)。模型吃 ID,不吃文本。

ID 就是 one-hot 的紧凑表示

token ID = 856,等价于一个 32006 维的 one-hot(第 856 位是 1,其余 0)。**词表大小 32006 = one-hot 维度**。但实际存/算用整数 ID(省空间),不展开成稀疏 one-hot。

embedding = one-hot × 嵌入矩阵 = 取矩阵第 856 行。所以 `nn.Embedding` 直接"按 ID 索引取行",跳过 one-hot 的稀疏乘法(等价但快)。

embedding 是学出来的(不是现成的)

embedding 表($32006 \times d_{\text{model}}$)一开始随机初始化,跟着 **next-token 任务端到端学**——不是 word2vec 那种独立预训练固定的词向量。CE 的梯度经 BP 传回 embedding 表,更新每个(出现过的)token 的向量。

思想对应:这和 REALM 的"检索器表示随下游任务端到端学"是同一思想——表示不独立固定,而是被最终任务塑造。你 ranking 里两塔 embedding 被排序任务端到端塑造,也是这个。

5. 模型:0.6B decoder

组件	选择	说明
规模	d=1280, L=32, heads=20	675M,深(32层)利于多步推理

归一化	RMSNorm	比 LayerNorm 简单、稳
位置编码	RoPE	无参数,context 变化不增参数
FFN	SwiGLU	$\text{ffn_dim} \approx 2.67 \times d$, 现代标配
attention	SDPA(FlashAttention)	不存 $n \times n$ 矩阵, 省显存
其他	tie_embeddings, 残差按层缩放初始化	省参数, 稳定深层训练

参数 vs 激活 vs input 长度(三者关系):参数($W_q/W_k/W_v/\text{FFN}$)只由 d_{model} 决定, 和 **seq 长度无关** (所有 token 共享同一套参数, 复用)。seq 变长只增激活($n \times n$ attention 矩阵、每 token 的中间张量), 不增参数。"32 层深"能训得动, 靠的是**残差连接**(每层 $x + f(x)$), 梯度直穿。

6. 训练机制:CE、BP、梯度累积

前向 → CE → 反向

模型对每个位置输出 32006 维分布(softmax)。CE loss 逐位置算"正确 token 的 $-\log$ 概率", 再对所有位置平均。BP 把梯度经链式法则一路传回, 更新所有参数——包括 embedding 表(只更新 batch 里出现过的 token 那些行, 稀疏更新)。

micro-batch vs effective batch(梯度累积)

micro_batch = 一次 forward 喂 GPU 的条数(受显存限, 大了 OOM)。

effective batch = 真正更新一次参数的总量(要大才稳, 如 0.5M token/step)。

显存装不下大 batch → **梯度累积**: 累积 N 次 micro 的梯度再更新一次。

$\text{eff_batch} = \text{micro_batch} \times \text{seq} \times \text{grad_accum}$ 。调 micro_batch(防 OOM)不改 eff_batch(grad_accum 自动补)——只影响显存和快慢, 不影响训练效果。

step vs epoch: 1 step = 一个 effective batch(0.49M token)更新一次 = 数据的 $1/28000$ 。训完整 14B 数据 1 遍(1 epoch) = $14\text{B} / 0.49\text{M} = 28000$ **step**。所以 $\text{total_steps} = 28000$ = 训 1 遍全量。

7. Smoke test 的第一性推导: 初始 $\text{loss} = \log|\text{vocab}|$

训练启动第一件事: 看 step 0 的 loss 是不是 $\approx \log|\text{vocab}|$ ——这是验证"模型正确初始化、无 bug"的标准探针。推导:

$$L = -(1/N) \sum_i \sum_c y_{\{i,c\}} \cdot \log \hat{y}_{\{i,c\}}$$

两个部件:

- **label y 是 one-hot**: 每个位置只有正确 token 那一维是 1, 其余 0 → 内层对 c 的求和塌缩成只剩正确 token 那一项 $-\log \hat{y}_{\{i, \text{正确}\}}$ 。
- **随机初始化 → 预测 $\hat{y}$ 近似均匀**: 权重随机(std 小) → logits ≈ 0 → softmax $\approx$ 均匀 → 每维 $\approx 1/|\text{vocab}|$ 。

代入(均匀下正确 token 概率 = $1/|\text{vocab}|$, 每个位置都一样, 平均后不变):

$$L = -(1/N) \sum_i \log(1/|\text{vocab}|) = \log|\text{vocab}|$$

实测验证: vocab=32006 → $\log(32006) \approx 10.37$ 。训练 step 0 loss = **10.63** ≈ 10.37 ✓。模型正确初始化(均匀瞎猜), 无 bug, 从随机基线开始学。若远大于 → 初始化有问题; 若远小于 → 数据泄漏/label 错位。

8. 困惑度: "平均在几个词里选"

困惑度 perplexity = e^{CE} 。由第 7 节, 随机时:

$$\text{ppl} = e^L = e^{\{\log|\text{vocab}|\}} = |\text{vocab}|$$

更本质的直觉——把均匀分布的词数设为 k:

$$\text{均匀分布在 } k \text{ 个词上} \rightarrow \text{CE} = \log k \rightarrow \text{ppl} = e^{\{\log k\}} = k$$

所以 ppl = "模型的不确定性, 相当于在几个词里均匀犹豫"(等效候选数)。

- 随机初始化: $\text{ppl} = |\text{vocab}| = 32006$ (在全词表 3.2 万个里瞎猜)。
- 训练后 $\text{ppl} = 4.8$: 不确定性相当于"在 ~5 个词里选"——很确定了。
- **越小越好**: 从"3.2万个候选"缩到"5个候选" = 模型学到了 C++ 的强规律。

代码的 ppl 天然低(个位数)——语法规律性强, 比自然语言(十几到几十)可预测得多。

9. 显存 back-and-forth: 激活、grad checkpointing、OOM

这一段是反复试错定位的过程, 值得记录——核心矛盾: 大 context + 深模型的激活太大, 和"关 grad_ckpt 提速"冲突。

OOM 反复(关 grad_ckpt 下)

配置(关 grad_ckpt)	显存	结果
8K context, micro16	78.5G	OOM

8K, micro8	79.2G	OOM
4K, micro12	79.1G	OOM
4K, micro8	78.6G	OOM

误判 + 定位:一开始以为降 micro_batch 能解决,越降越试越 OOM。真因是——**关 grad_ckpt 时 32 层激活全存,基数太大(60-70G),micro_batch 再小也爆**。SwiGLU 中间维 3456,单层激活就 ~1G,32 层 ~38G,加 attention/临时张量/碎片 → 逼近 80G。

激活显存的精确估算

每层激活 $\approx b \times S \times (4 \times d + 3 \times \text{ffn} + 2 \times d) \times 2$ 字节。b=8,S=4096,d=1280,ffn=3456:

每层 $\approx 8 \times 4096 \times 18048 \times 2 \approx 1.18\text{G} \rightarrow 32 \text{ 层} \approx \mathbf{38\text{G}}$ 。

+ 固定开销(参数1.35 + 梯度1.35 + Adam-FP32 5.4 $\approx 8\text{G}$) + 临时张量 + PyTorch caching/碎片 ($\times 1.5-1.7$) $\rightarrow$ 实测 ~78G,临界爆。

理论 vs 实测差 1.7 倍:PyTorch caching allocator 预留 + 显存碎片 + 临时张量峰值 + 激活低估。深度学习显存理论只估数量级,精确要实测。

结论:大 context + 深模型必须开 grad_checkpointing

grad_checkpointing:前向只存少数检查点层的激活,反向时从最近检查点**重算**中间激活(用完即丢)。省的是"激活存储"(靠重算),**不是梯度**(梯度都算),**不是"只留最后一层"**。

- 关 gckpt:32 层激活全存 $\rightarrow \sim 70\text{G} \rightarrow$ 爆。
- 开 gckpt:激活不全存 $\rightarrow \sim 30-40\text{G} \rightarrow$ micro24 也不爆。

代价:反向重算前向,慢 ~25%。**但对深模型+长 context 是硬需求,不是可选**——关它反而把 batch 逼到很小还 OOM,不划算。

精度的影响:训练用 BF16(参数/梯度/激活 2 字节),只有 Adam 状态 FP32(4字节,要高精度累积)。若全 FP32,激活翻倍(38 $\rightarrow$ 76G),爆得更狠。BF16 是省显存的关键。

反思:80G 其实浪费:开 gckpt 后显存需求才 30-40G,**40G 卡就够**。用到 80G 是"关 gckpt 硬凑大 batch"的错误方向——不是真需要。

10. 速度 back-and-forth:GPU 利用率、异步预取

现象:GPU 大部分 0%,tok/s 卡在 17k

开 gckpt 跑起来了,但 `nvidia-smi` 抓到 GPU 大部分 0%,tok/s 只有 17k(14B 要 ~9 天)。

定位:数据加载同步阻塞

单独测 `get_batch`:2392k tok/s(数据本身飞快)。但训练时 GPU 0% → **不是数据慢,是同步阻塞**:GPU 算完一步,停下等 CPU 采下一批(`get_batch`)+ pin + 传输,GPU 空转。`grad_accum` 每步采多次,间隙更多。

解法:异步预取(流水线重叠)

后台线程持续采 batch + pin,放入队列;主循环直接从队列拿(已备好);CUDA stream 异步传输。
让"CPU 采数据"和"GPU 计算"重叠(流水线),GPU 不空等。这是 HFT/并发里的生产者-消费者 + 双缓冲思想,用在训练数据加载上。
效果:GPU 利用率 0% → **100%**。

但——预取后真瓶颈暴露:GPU 100% 了,tok/s 仍 ~18k(几乎没涨)。说明数据加载不是主瓶颈,**`grad_checkpointing` 的重算才是**——GPU 100% 是"忙于重算激活",有效吞吐被重算吃掉。真提速需减重算:减 context 到 2K(激活小,可关 `gckpt` 不重算)+ 预取,可达 40k+。这是"先跑通攒 warm start,再优化"的取舍——先用 4K+`gckpt` 稳定训出 checkpoint,后续换 2K 提速续训。

另一个拖速度的坑:ckpt_every。smoke test 把 `ckpt_every` 改成 10 未改回,每 10 步存一个 8.1G checkpoint,存盘 IO 拖到每步 55 秒(一夜才 790 步)。改回 100 后,存盘频率降 10 倍,tok/s 回到 18k。**`keep_last_k=3` 一直生效**(只留最近 3 个 + best + last),没堆爆盘。

11. 结果:陡降与平台期

loss 从 10.63 → 1.54,ppl 从 41569 → 4.7(step ~800,仅训 0.4B token,数据的 3%)。gnorm 稳定在 0.4-1.4(健康,无爆炸)。

loss 曲线两阶段:

- **陡降期**(前期,几百步):快速学基础规律(语法、括号配对、常见模式),规律强好学,loss 陡降到 ~1.5。
 - **平台期**(之后):在 1.5 附近缓降(磨复杂逻辑、长尾模式),边际递减,"1.5 会持续很久"。
- 所以陡降到 **1.5 = 基本 C++ 会写了**(阶段 A 核心目标)。判断"够不够"用生成测试(会不会写像样 C++),而非死等 loss——平台期 loss 降不动,但生成可能已够好。

阶段 A 小结

从零(随机初始化)手写 Transformer,在 14B C++ token 上预训练,**~3% 数据即达 loss 1.5 / ppl 4.7,会写基本 C++**。全链路自己搭:数据清洗(32 核并行)、混合 tokenizer、0.6B decoder、训练循环(梯

度累积/断点续训/异步预取)。踩透了显存(激活/gckpt/OOM)、速度(利用率/预取/真瓶颈)的工程取舍,并从第一性原理推穿了 $CE = \log|\text{vocab}|$ 与困惑度 = 等效候选数。

— 阶段一完。下一站:B(SFT 题解对齐)+ C(RLVR 强化真解题)。 —